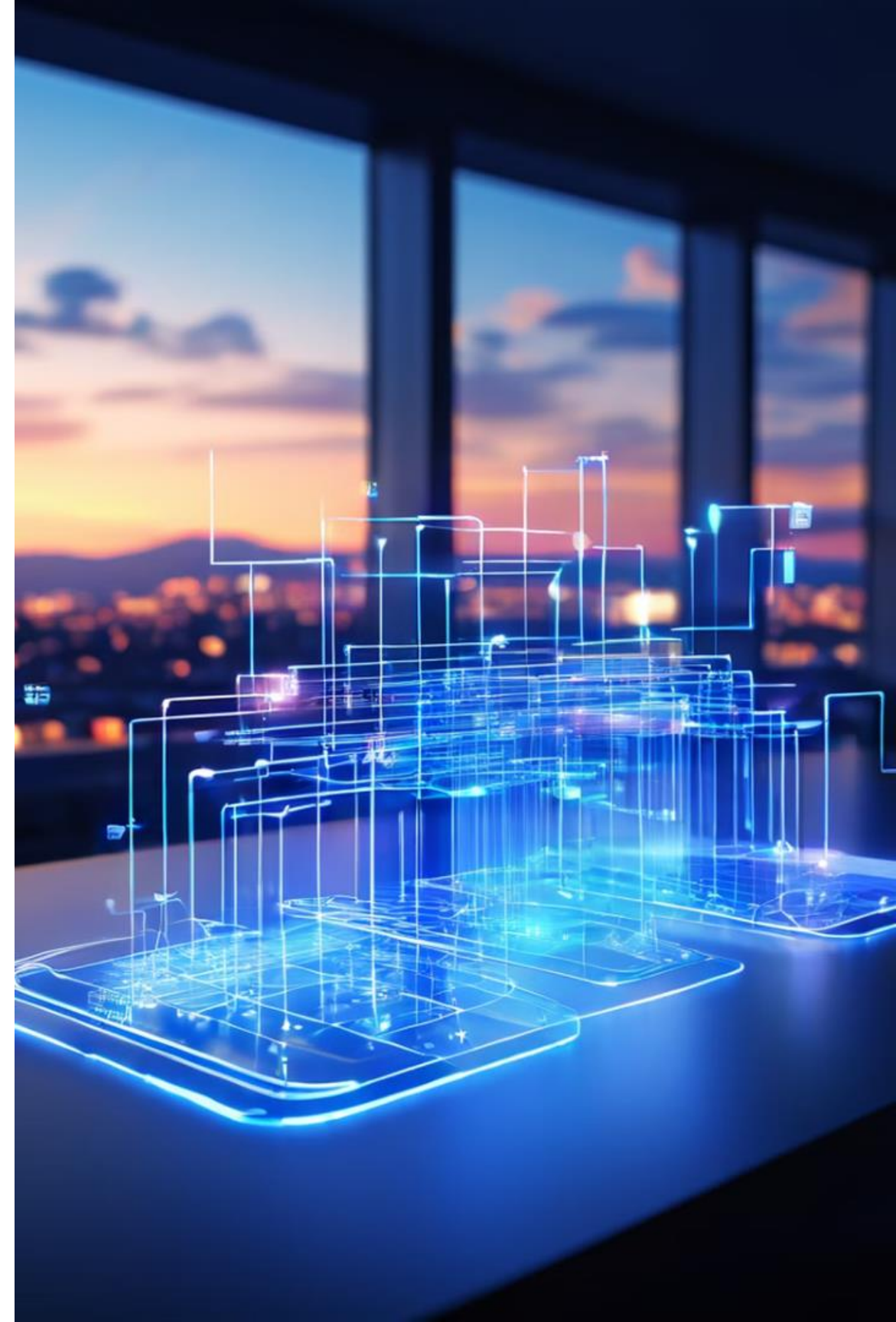


Views no SQL: Conceitos, Aplicações e Benefícios

Uma abordagem detalhada sobre como as Views no MySQL podem simplificar consultas, aumentar a segurança e facilitar a manutenção do seu banco de dados.

 por Salvador Melo





Introdução

As Views (ou "Visões") são uma forma de criar tabelas virtuais no MySQL, baseadas em consultas SQL. Elas permitem encapsular consultas complexas, facilitando o acesso a dados e melhorando a organização do banco de dados.

Neste material, exploraremos:

- O que são views e como criá-las
- Views atualizáveis e inseríveis
- Uso do WITH CHECK OPTION
- Restrições das views
- Algoritmos de processamento de views
- Exemplos práticos com o Sistema Acadêmico

O que são Views no MySQL?

📌 Conceito

Uma View no MySQL é uma tabela virtual baseada no resultado de uma consulta SQL. Ela permite que você crie uma representação simplificada dos dados armazenados em múltiplas tabelas, facilitando a consulta e melhorando a segurança.

🏠 Exemplo Prático:

Quantos alunos estão matriculados em cada disciplina e quem é o professor responsável?

Usar:

```
SistemaAcademico.sql
```



Consulta Direta

```
SELECT
    d.nome AS Disciplina,
    p.nome AS Professor,
    COUNT(m.id_estudante) AS Total_Alunos
FROM
    Matriculas m
    JOIN
    Disciplinas d ON m.id_disciplina = d.id_disciplina
    JOIN
    Professores p ON d.id_professor = p.id_professor
GROUP BY d.nome , p.nome
ORDER BY Total_Alunos DESC;
```

Resultado:

Disciplina	Professor	Total_Alunos
Banco de Dados	Dr. Ricardo Silva	2
Algoritmos	Dr. Marcos Ribeiro	2
Estruturas de Dados	Profa. Juliana Costa	1



Criando uma view

```
CREATE VIEW alunos_por_disciplina AS
SELECT
    d.nome AS Disciplina,
    p.nome AS Professor,
    COUNT(m.id_estudante) AS Total_Alunos
FROM
    Matriculas m
    JOIN
    Disciplinas d ON m.id_disciplina = d.id_disciplina
    JOIN
    Professores p ON d.id_professor = p.id_professor
GROUP BY d.nome , p.nome
ORDER BY Total_Alunos DESC;
```

select * from alunos_por_disciplina;

Disciplina	Professor	Total_Alunos
Banco de Dados	Dr. Ricardo Silva	2
Algoritmos	Dr. Marcos Ribeiro	2
Estruturas de Dados	Profa. Juliana Costa	1



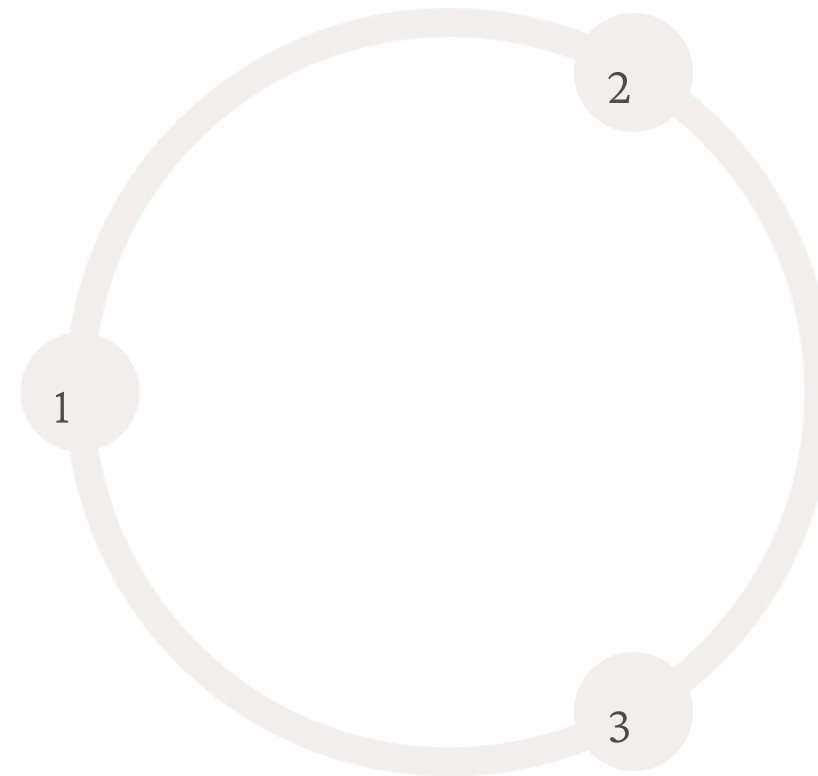
Diferenças entre Tabelas e Views

Característica	Tabela	View
Armazena dados?	✓ Sim	✗ Não
Pode ser usada em consultas?	✓ Sim	✓ Sim
Pode ser modificada diretamente?	✓ Sim	⚠ Parcialmente (depende do conteúdo da View)

Benefícios das Views

Simplicidade

As Views encapsulam consultas SQL complexas, tornando mais fácil a reutilização do código.



Segurança

As Views restringem o acesso a dados sensíveis ao permitir que usuários vejam apenas colunas específicas.

Facilidade de Manutenção

Se as colunas das tabelas mudarem, podemos simplesmente atualizar a View em vez de modificar todas as consultas do sistema.

Benefício: Simplicidade

🎯 Simplicidade

As Views encapsulam consultas SQL complexas, tornando mais fácil a reutilização do código.

Caso Real

No RH de uma empresa, um relatório mensal precisa calcular a média salarial de cada departamento. Em vez de reescrever essa consulta toda vez, criamos uma View.





Implementação da View para Simplicidade

1

Criação da View

CREATE VIEW

view_media_salarial

AS SELECT

departamento,

AVG(salario) AS media_salarial

FROM

Funcionarios

GROUP BY

departamento;

2

Consulta Segura e Restrita

Em um sistema de gestão de recursos humanos, um desenvolvedor que está construindo um painel para os gestores pode precisar exibir a média salarial por departamento, mas sem acesso direto aos dados individuais dos funcionários. Com uma view, ele pode consultar essas informações sem permissão para acessar toda a base de dados.

3

Resultado

SELECT * FROM view_media_salarial;

Isso garante que ele obtenha os dados necessários sem visualizar salários individuais, preservando a privacidade das informações.

Benefício: Segurança



Segurança

As Views restringem o acesso a dados sensíveis ao permitir que usuários vejam apenas colunas específicas.



Caso Real

Em um sistema bancário, a tabela de contas contém informações de todos os clientes, incluindo saldos e dados financeiros sensíveis. O problema não está no usuário final, que interage com a aplicação por meio de uma interface segura, mas sim no desenvolvedor backend, que pode precisar acessar o banco de dados para implementar funcionalidades.

Benefício: Facilidade de Manutenção

✂ Facilidade de Manutenção

Se as colunas das tabelas mudarem, podemos simplesmente atualizar a View em vez de modificar todas as consultas do sistema.

Caso Real

Em um e-commerce, os analistas de dados consultam frequentemente os produtos mais vendidos. Em vez de criar novas consultas sempre que precisam, usamos uma View.



Implementação da View para Manutenção

Criação da View

```
CREATE VIEW view_top_produtos AS SELECT produto, COUNT(*) AS  
total_vendas FROM Vendas GROUP BY produto ORDER BY  
total_vendas DESC;
```

Acesso Simplificado

A equipe de análise pode acessar esses dados sem precisar entender SQL avançado.

Atualização Centralizada

Se mudarmos a estrutura das tabelas, atualizamos apenas a definição da View.





Removendo uma View

1

Identificar Views Obsoletas

Primeiro, identifique Views que não são mais utilizadas.

2

Validar Dependências

Verifique se outras Views ou sistemas dependem da View a ser removida.

3

Executar o Comando

✦ 4.4 Removendo uma View: DROP

```
VIEW nome_da_view;
```


Removendo uma View

Para excluir uma view do banco de dados, utilizamos o comando DROP VIEW, que remove uma ou mais visualizações do sistema.

Sintaxe do Comando

```
DROP VIEW [IF EXISTS]  
view_name [, view_name] ...
```

- IF EXISTS - Evita que ocorra um erro caso a view não exista
- view_name - Nome da view ou lista de views a serem removidas

É necessário ter o privilégio DROP para cada visualização que deseja remover. Se alguma visualização na lista não existir e a cláusula IF EXISTS não for utilizada, a instrução falhará com uma mensagem de erro.

Variação de Comportamento entre MySQL 8.3 e 8.4

- No MySQL 8.3 e anteriores, DROP VIEW remove as views existentes e ignora as que não existem.
- No MySQL 8.4, a instrução falha completamente se alguma das views não existir.

 **Solução:** Sempre usar IF EXISTS para evitar falhas inesperadas.

Exemplo de DROP VIEW

Suponha que temos as seguintes views no banco Sistema Acadêmico:

```
CREATE VIEW ViewAlunosTI AS  
SELECT id, nome, curso FROM Aluno WHERE curso = 'Tecnologia da Informação';  
  
CREATE VIEW ViewAlunosGraduacao AS  
SELECT id, nome FROM Aluno WHERE curso IN ('Engenharia', 'Ciência de Dados');
```

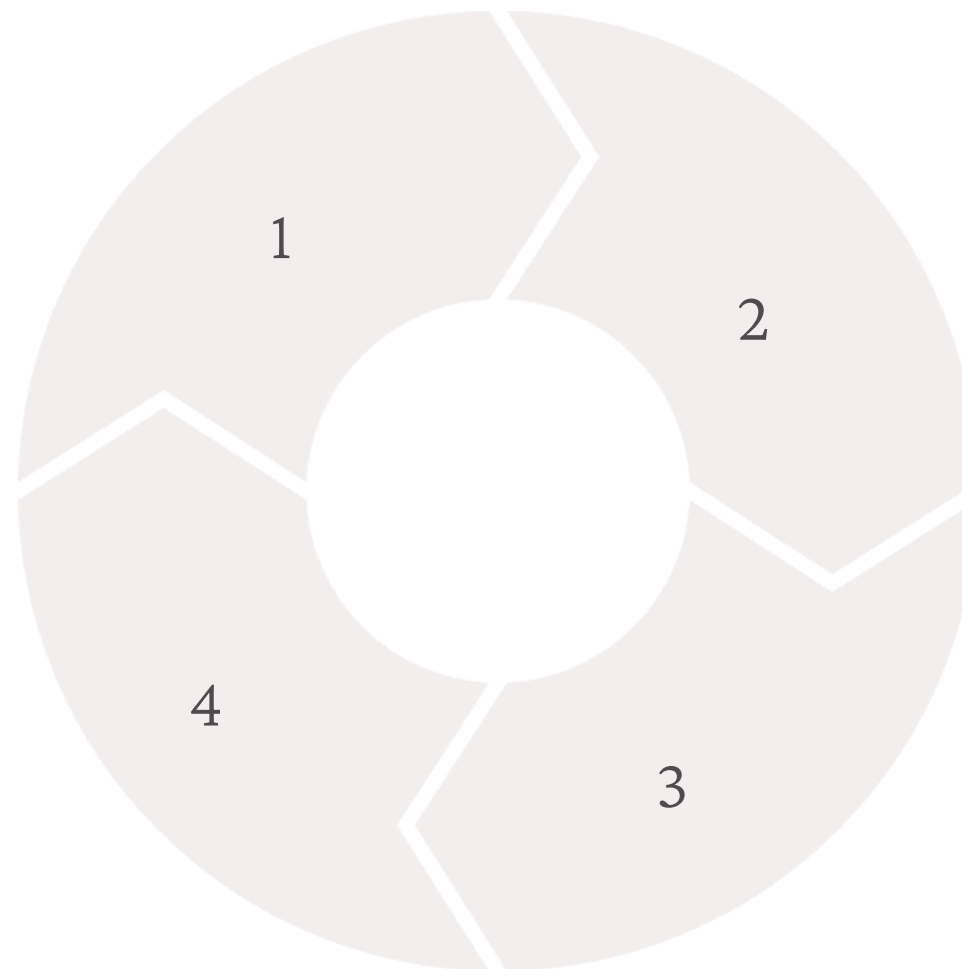
Se quisermos remover essas views, podemos usar:

```
DROP VIEW IF EXISTS ViewAlunosTI, ViewAlunosPosGraduacao;
```

Criação e Uso de Views

Criação
Definir a estrutura da View

Remoção
Excluir quando obsoleta



Consulta
Utilizar a View como tabela

Alteração
Modificar quando necessário

Atualização de Views no MySQL

📌 O que significa atualizar uma View?

Atualizar uma *View* no MySQL refere-se a modificar sua definição, ou seja, alterar a consulta subjacente que define os dados exibidos pela *View*. Isso é feito quando precisamos incluir novas colunas, modificar filtros ou ajustar a lógica da consulta, sem a necessidade de remover e recriar a *View* manualmente.

🚀 Por que atualizar uma View em vez de criar uma nova?

1. Facilidade de manutenção – Se a *View* já está sendo usada por várias partes do sistema, é melhor alterá-la do que criar uma nova e modificar todo o código que a utiliza.
2. Menos impacto no sistema – Alterar uma *View* evita que outras consultas que dependem dela quebrem, pois seu nome continua o mesmo.
3. Refinamento de requisitos – Se um relatório ou dashboard precisa de mais informações, podemos simplesmente alterar a *View* existente sem reescrever consultas em várias partes do sistema.

Alterar uma view existente no MySQL

1. Alterar uma View com ALTER VIEW

Suponha que temos a seguinte *View*, que exibe o nome dos alunos e seus cursos:

```
CREATE VIEW view_alunos_cursos AS  
SELECT nome, curso FROM Alunos;
```

Agora, o setor acadêmico pediu para incluir a idade dos alunos na *View*. Em vez de recriar a *View*, usamos ALTER VIEW para modificá-la:

```
ALTER VIEW view_alunos_cursos AS  
SELECT nome, curso, idade FROM Alunos;
```

Agora, qualquer consulta a `view_alunos_cursos` incluirá a idade dos alunos automaticamente.

Testar:

`universidade.sql`

Alterar uma view existente no MySQL

2. Outra Opção: CREATE OR REPLACE VIEW

Suponha que temos a seguinte *View*, que exibe o nome dos alunos e seus cursos:

```
CREATE VIEW view_alunos_cursos AS  
SELECT nome, curso FROM Alunos;
```

Agora, o setor acadêmico pediu para incluir a idade dos alunos na *View*. Em vez de recriar a *View*, usamos `ALTER VIEW` para modificá-la:

```
CREATE OR REPLACE VIEW view_alunos_cursos AS  
SELECT nome, curso, idade FROM Alunos;
```

Agora, qualquer consulta a `view_alunos_cursos` incluirá a idade dos alunos automaticamente.

Testar: universidade.sql

Diferenças principais:

- **Privilégios:** `ALTER VIEW` mantém os privilégios associados à view, enquanto `CREATE OR REPLACE VIEW` redefine a view e todos os privilégios são perdidos.
- **Flexibilidade:** `ALTER VIEW` é mais adequado para alterações incrementais, enquanto `CREATE OR REPLACE VIEW` é útil para redefinir completamente a view.

❑ Quando usar `ALTER VIEW` e quando recriar a View?

Situação	Solução
Preciso adicionar/remover colunas da mesma tabela na <i>View</i>	✓ <code>ALTER VIEW</code>
Quero modificar filtros da <i>View</i> (ex: <code>WHERE idade > 18</code> para <code>WHERE idade > 21</code>)	✓ <code>ALTER VIEW</code>
Preciso adicionar colunas de outra tabela usando <code>JOIN</code>	⚠❑ Necessário recriar a View
A <i>View</i> contém <code>GROUP BY</code> , <code>DISTINCT</code> ou funções agregadas	⚠❑ Necessário recriar a View
Quero modificar completamente a lógica da <i>View</i>	⚠❑ Melhor recriar a View

Operações de INSERT, UPDATE e DELETE podem ser aplicadas (ou não) usando VIEWS

Tipo de View	INSERT	UPDATE	DELETE	Observações
View Simples (Uma Tabela)	✔ Possível	✔ Possível	✔ Possível	Todas as colunas NOT NULL da tabela base devem ser atendidas
View com Junção (Joins)	✘ Geralmente não	✘ Geralmente não	✘ Geralmente não	Operações podem ser ambíguas ou complexas para o MySQL resolver
View com Agregação (GROUP BY, COUNT, etc.)	✘ Não permitido	✘ Não permitido	✘ Não permitido	Resultados derivados não são diretamente modificáveis

As restrições variam conforme o tipo de View, sendo que apenas Views simples (baseadas em uma única tabela) geralmente permitem operações de modificação direta.

Exemplo - Uma Tabela(usando SistemaAcademico.sql)

id_estudante	nome	email
1	Alice Souza	alice@email.com
2	Bruno Lima	bruno@email.com
3	Carlos Mendes	carlos@email.com

id_estudante	nome	email
1	Alice Souza	alice@email.com
2	Bruno Lima	bruno@email.com
3	Carlos Mendes	carlos@email.com
7	Daniel Pereira	daniel@email.com

Vamos ver como as operações com views funcionam na prática usando nosso banco de dados SistemaAcademico.

```
drop view ViewEstudantes;
-- Criando uma view simples dos estudantes
CREATE VIEW ViewEstudantes AS
SELECT id_estudante, nome, email
FROM Estudantes;

select * from ViewEstudantes;

-- INSERT (Adicionando um novo estudante)
INSERT INTO ViewEstudantes (nome, email) VALUES ('Daniel Pereira', 'daniel@email.com');
select * from ViewEstudantes;

-- UPDATE (Atualizando o email de um estudante)
UPDATE ViewEstudantes SET email = 'dan_pereira@email.com' WHERE nome = 'Daniel Pereira';
select * from ViewEstudantes;

-- DELETE (Removendo um estudante)
DELETE FROM ViewEstudantes WHERE nome = 'Daniel Pereira';
select * from ViewEstudantes;
```

Esta view simplifica consultas frequentes, combinando informações de alunos e suas notas em uma única estrutura.

Nem sempre pode - Exemplo

```
drop view alunos_por_disciplina;
-- criando uma view
CREATE VIEW alunos_por_disciplina AS
SELECT
    d.nome AS Disciplina,
    p.nome AS Professor,
    COUNT(m.id_estudante) AS Total_Alunos
FROM
    Matriculas m
    JOIN
    Disciplinas d ON m.id_disciplina = d.id_disciplina
    JOIN
    Professores p ON d.id_professor = p.id_professor
GROUP BY d.nome , p.nome
ORDER BY Total_Alunos DESC;

select * from alunos_por_disciplina;

-- View não atualizável
insert into alunos_por_disciplina(d.nome) values('Cálculo1');
```

08:54:48 insert into alunos_por_disciplina(d.nome)
values('Cálculo1')

Error Code: 1471. The target table alunos_por_disciplina of
the INSERT is not insertable-into 0.000 sec

Views no MySQL Sobrecarregam as Consultas?

📌 O impacto das Views no desempenho

As *Views* no MySQL não armazenam dados, apenas exibem os resultados de uma consulta predefinida. Isso significa que toda vez que uma View é consultada, o MySQL executa novamente a consulta original. Dependendo da complexidade da consulta e do volume de dados, isso pode sim sobrecarregar o banco de dados.

⚖️📦 Quando Views são eficientes?

✓ 1. Para simplificar consultas repetitivas

Se uma consulta complexa precisa ser executada frequentemente, encapsular essa lógica em uma *View* evita que os desenvolvedores precisem reescrever a consulta toda vez.

✓ 2. Para restringir acesso a dados sensíveis

Em sistemas onde diferentes usuários têm diferentes níveis de permissão, uma *View* pode fornecer acesso a informações restritas sem expor colunas sensíveis da tabela original.

✓ 3. Para organizar relatórios e dashboards

Relatórios podem exigir consultas elaboradas com *joins* e cálculos. Usar uma *View* permite padronizar a estrutura dos dados exibidos sem exigir que usuários finais dominem SQL.

✗ Quando Views podem ser um problema?

🕒 1. Views são executadas em tempo real

Como não armazenam dados, sempre que uma *View* é consultada, o banco precisa processar novamente toda a consulta original. Se a *View* contém *joins* complexos ou grandes agregações, isso pode impactar o desempenho.

🕒 2. Views não são otimizadas automaticamente

O MySQL não cria automaticamente índices para *Views*. Isso significa que, se uma *View* faz uma junção entre tabelas grandes, pode ser mais lento do que executar a consulta diretamente sobre tabelas bem indexadas.

🕒 3. Views aninhadas podem ser problemáticas

Se uma *View* é baseada em outra *View* que já faz múltiplos *joins*, o MySQL pode executar consultas extremamente complexas sem otimização eficiente. Isso pode gerar consultas muito lentas.

E como saber as views existentes?

Para listar todas as views existentes no banco de dados, podemos consultar a tabela `INFORMATION_SCHEMA.VIEWS`, que faz parte do dicionário de dados do MySQL.

1. Listar Todas as Views do Banco

A seguinte consulta retorna todas as views existentes no banco de dados atual:

```
SELECT TABLE_SCHEMA, TABLE_NAME  
FROM INFORMATION_SCHEMA.VIEWS;
```

✦ Explicação:

- `TABLE_SCHEMA` → Nome do banco de dados onde a view está armazenada.
- `TABLE_NAME` → Nome da view.

TABLE_SCHEMA	TABLE_NAME
faculdade	view_alunos_cursos
faculdade	view_alunos_professores
hospital	view_pacientes_contato
hospital	view_pacientes_financeiro
hospital	view_pacientes_historico

E como saber as views existentes?

2. Listar Views de um Banco Específico

Se quisermos listar apenas as views de um banco específico (exemplo: SistemaAcademico), usamos:

```
SELECT TABLE_NAME  
FROM INFORMATION_SCHEMA.VIEWS  
WHERE TABLE_SCHEMA = 'SistemaAcademico';
```

✦ Isso mostrará apenas as views do banco SistemaAcademico.

TABLE_NAME
alunos_por_disciplina



Boas Práticas para Views

1 Nomenclatura Clara

Use prefixos como "view_" para identificar facilmente objetos que são Views.

3 Desempenho

Evite criar Views baseadas em outras Views para não comprometer o desempenho.

2 Documentação

Documente o propósito de cada View e suas limitações para outros desenvolvedores.

4 Manutenção

Revisão periódica para identificar Views obsoletas ou que precisam ser atualizadas.